Búsqueda Binaria/Ternaria

ICPC Campamento de Programación Oruro 2024

Adivina mi número

Estoy pensando en un número entre 1 y 1 millón, tienes que adivinar cual es, cuando me digas un número yo te diré si es que el número en el que estoy pensando es menor o no al número que me dijiste.

Binary Search Tutoriales

https://usaco.guide/silver/binary-search?lang=cpp

https://codeforces.com/edu/course/2/lesson/6

Encontrar número en arreglo ordenado

Dado un arreglo **ordenado** de enteros, indicar si contiene un elemento **Val** o no.

- Revisamos el elemento del medio
  - Si es **Val**, terminamos
  - Si es mayor que **Val**, buscamos en la mitad izquierda
  - Si es menor que **Val**, buscamos en la mitad derecha
- Hacemos lo mismo en las mitades más pequeñas hasta encontrar el número o hasta que no queden más elementos
- Primero hay n candidatos, luego n/2 , luego n/4 , etc.
- O(log n)
- Hay una función ya implementada

Encontrar número en arreglo ordenado

Val = 22

1	4	5	8	11	19	21	22	25	29	31	37	42
lo						m						hi

Encontrar número en arreglo ordenado

Val = 22

1	4	5	8	11	19	21	22	25	29	31	37	42
×	×	×	×	×	×	×	lo		m			hi

Encontrar número en arreglo ordenado

Val = 22

1	4	5	8	11	19	21	22	25	29	31	37	42
×	×	×	×	×	×	×	lo m	hi	×	×	×	×

Encontrar número en arreglo ordenado

Val = 22

1	4	5	8	11	19	21	22	25	29	31	37	42
×	×	×	×	×	×	×	lo	hi	×	×	×	×
							m					

Encontrar número en arreglo ordenado

Val = 23

1	4	5	8	11	19	21	22	25	29	31	37	42
lo						m						hi

Encontrar número en arreglo ordenado

Val = 23

1	4	5	8	11	19	21	22	25	29	31	37	42
×	×	×	×	×	×	×	lo		m			hi

Encontrar número en arreglo ordenado

Val = 23

1	4	5	8	11	19	21	22	25	29	31	37	42
×	×	×	×	×	×	×	lo m	hi	×	×	×	×

Encontrar número en arreglo ordenado

Val = 23

1	4	5	8	11	19	21	22	25	29	31	37	42
×	×	×	×	×	×	×	×	lo hi m	×	×	×	×

Encontrar número en arreglo ordenado

Val = 23

1	4	5	8	11	19	21	22	25	29	31	37	42
×	×	×	×	×	×	×	×	×	×	×	×	×
							hi	lo				

Encontrar número en arreglo ordenado

Val = 23

1	4	5	8	11	19	21	22	25	29	31	37	42
×	×	×	×	×	×	×	×	×	×	×	×	×
							hi	lo				

Función Monótona

Sea: **f : P -> Q** una función entre dos conjuntos **P** y **Q**, donde cada conjunto tiene un orden.

La función f es **monótona** si y sólo si x <= y implica f(x) <= f(y) (es decir, la función es **creciente**), o bien x <= y implica f(x) >= f(y) (es decir, la función es **decreciente**). En otras palabras, una función es monótona si conserva el orden.

Es decir una función es **monótona** cuando es *creciente* o *decreciente* en todo su dominio.

Forma general

- Dado f(x) que devuelve 1 o 0
- f(x) = 1 si x cumple la propiedad
- f(x) = 0 si x no cumple la propiedad
- f(x) es monótona

Val = 22

f(x) = 1 si x >= Val

f(x) = 0 si x < Val

1	4	5	8	11	19	21	22	25	29	31	37	42
0	0	0	0	0	0	0	1	1	1	1	1	1

Se puede buscar el primer 1 o el último 0

Forma general: encontrar el primer uno

```
int lo = 0, hi = n-1;
int res = -1;
while (lo <= hi) {
    int mid = (lo + hi) / 2;
    if (f(mid) == 1) {
        res = mid; // mejor respuesta hasta ahora
        hi = mid - 1;
    }
    else { // f(mid) == 0
        lo = mid + 1;
    }
}
// Primer 1 esta en la posicion res
```

Forma general: encontrar el último cero

```
int lo = 0, hi = n-1;
int res = -1;
while (lo <= hi) {
    int mid = (lo + hi) / 2;
    if (f(mid) == 0) {
        res = mid; // mejor respuesta hasta ahora
        lo = mid + 1;
    }
    else { // f(mid) == 1
        hi = mid - 1;
    }
}
// Primer 1 esta en la posicion res
```

Forma general: encontrar el primer uno

Val = 25

1	4	5	8	11	19	21	22	25	29	31	37	42
lo						m						hi

Forma general: encontrar el primer uno

Val = 25

1	4	5	8	11	19	21	22	25	29	31	37	42
×	×	×	×	×	×	×	lo		m			hi
									res			

Forma general: encontrar el primer uno

Val = 25

1	4	5	8	11	19	21	22	25	29	31	37	42
×	×	×	×	×	×	×	lo	hi	×	×	×	×
							m					
									res			

Forma general: encontrar el primer uno

Val = 25

1	4	5	8	11	19	21	22	25	29	31	37	42
×	×	×	×	×	×	×	×	lo	×	×	×	×
								hi				
								m				
								res	~~res~~			

Forma general: encontrar el primer uno

Val = 25

1	4	5	8	11	19	21	22	25	29	31	37	42
×	×	×	×	×	×	×	×	×	×	×	×	×
							hi	lo				
								res	~~res~~			

Forma general

Igual se puede encontrar el último uno o el primer cero, depende de cómo esté modelada la función

- Encontrar el último uno (1, 1, 1, 1, 1, 0, 0, 0, 0)
- Encontrar el primer cero (1, 1, 1, 1, 1, 0, 0, 0, 0)
- Encontrar el primer uno (0, 0, 0, 0, 1, 1, 1, 1, 1)
- Encontrar el último cero (0, 0, 0, 0, 1, 1, 1, 1, 1)

Problema 1

Tenemos un arreglo a que contiene **n** enteros. Se deben responder **q** consultas de la forma: Dados dos valores **l** y **r**, ¿cuantos elementos del arreglo a están en el rango [l,r] (mayores o iguales a l y menores o iguales a r)?

- Ordenamos el arreglo
- Para encontrar límite izquierdo, buscamos el primer elemento mayor o igual que l
- Para encontrar el límite derecho, buscamos el último elemento menor o igual que r

Problema 1 código

```
// asumir lectura de n, ar, l, r y ar ordenado
int posl = -1, lo = 0, hi = n - 1;
while(lo <= hi) {
    int mid = (lo + hi) / 2;
    if(ar[mid] >= l)
        posl = mid, hi = mid - 1;
    else
        lo = mid + 1;
}
int posr = -1;
lo = 0, hi = n - 1;
while(lo <= hi) {
    int mid = (lo + hi) / 2;
    if(ar[mid] <= r)
        posr = mid, lo = mid + 1;
    else
        hi = mid - 1;
}
if(posl == -1 || posr == -1) {
    cout << 0 << '\n';
else
    cout << posr - posl + 1 << '\n';
```

Problema 1 código corto

```
upper_bound(ar.begin(), ar.end(), r) - lower_bound(ar.begin(), ar.end(), l)
```

- upper_bound(inicio, fin, valor, comparador_opcional)
- Te devuelve iterador al primer valor mayor a valor
- lower_bound(inicio, fin, valor, comparador_opcional)
- Te devuelve iterador al primer valor mayor o igual a valor.
- Para vectores o arreglos, si los restas te devuelven enteros
- Estas funciones igual hay en los sets y mapas, igual devuelven iteradores, pero NO SE LOS PUEDE RESTAR
- st.upper_bound(val), st.lower_bound(val)
- ma.upper_bound(val), ma.upper_bound(val)

Problema 2

Dados N números, decir cual es el número más cercano menor a X que se puede obtener sumando 2 números (se puede repetir el mismo número).

Ejemplo Entrada: N = 5, Array = [1, 3, 4, 7, 10], X = 12

Ejemplo Salida: 11 (4 + 7)

Solución:

- Ordenarlos
- Para cada número a_i hallar el número perteneciente al arreglo más grande que sea menor o igual a X - a_i
- La respuesta es el mayor
- O(nlog(n))

Problema 2 código

```
// asumir lectura de n, x y ar ordenado
int res = -1;
for(int i = 0; i < n; i++){
    auto it = upper_bound(ar.begin(), ar.end(), x - ar[i]);
    if(it != ar.begin())
        res = max(res, ar[i] + *--it);
}
```

Two pointers

- Técnica que utiliza dos índices o "punteros" para recorrer una estructura de datos (como un array o una cadena).
- Los punteros generalmente se mueven hacia el centro desde los extremos, o de izquierda a derecha.
- Usualmente se puede utilizar Binary Search pero Two Pointers es más rápido
- Aplicable a una amplia variedad de problemas, desde suma de pares, problemas con subarreglos y problemas de ventanas deslizantes.

Problema 2 solución two pointers

- Ordenar el los números.
- Fijar el puntero izquierdo.
- Iniciar un puntero derecho desde el final del arreglo.
- Mover el puntero derecho mientras la suma sea mayor a X
- Incrementar el puntero izquierdo y repetir el paso anterior.
- Cada vez antes de incrementar el puntero izquierdo, guardad la suma las alta.
- O(n)

Problema 2 código two pointers

```
// asumir lectura de n, x y ar ordenado
int res = -1, ri = n - 1;
for(int i = 0; i < n; i++){
    while(ri >= i && ar[i] + ar[ri] > x) ri--;
    if(ri >= i) res = max(res, ar[i] + ar[ri]);
}
```

Problema 3

Dados N números, decir cual es el número más cercano menor a X que se puede obtener sumando 3 números (se puede repetir el mismo número).

- Ordenar los números
- Fijar un puntero al medio
- Utilizar la técnica two pointers con las porciones de arreglo que separó el puntero al medio
- O(n^2)

Problema 3 código

```
// asumir lectura de n, x y ar ordenado
int res = -1;
for(int i = 1, i < n - 1; i++){
    int ri = n - 1;
    for(int j = 0; j < i; j++){
        while(ri > i && ar[i] + ar[j] + ar[ri] > x) ri--;
        if(ri > i) res = max(res, ar[i] + ar[j] + ar[ri]);
    }
}
```

Problema 4

Dado un arreglo de tamaño N, dividir el arreglo en K subarreglos minimizando la máxima suma de cada subarreglo

Ejemplo Entrada: N = 5, K = 3, Arreglo = [2, 4, 7, 2, 5]

Ejemplo Salida: 7

https://cses.fi/problemset/task/1085

Problema 4 solución

- Hacer Binary Search sobre el rango de posibles sumas máximas.
- Verificar si es posible dividir el arreglo con la suma máxima actual.

Función de verificación:

- Iterar sobre los elementos del arreglo sumando hasta que se exceda la suma máxima.
- Contar cuántos subarreglos se forman.
- Verificar si el número de subarreglos es menor o igual a K.

Problema 4 código

```
// asumir lectura n, k, ar
int lo = *max_element(all(ar)), hi = 1e9 * n, res = 0;
    while(lo <= hi) {
        int mid = (lo + hi) / 2;
        int sum = 0, cnt = 0;
        for(int i = 0; i < n; i++)
            if(sum + ar[i] <= mid)
                sum += ar[i];
            else
                cnt++, sum = ar[i];
        cnt++;
        if(cnt <= k)
            res = mid, hi = mid - 1;
        else
            lo = mid + 1;
    }
    cout<<res<<'\n';
```

Ternary Search Tutoriales

https://cp-algorithms.com/num_methods/ternary_search.html

https://codeforces.com/blog/entry/60702

Función unimodal

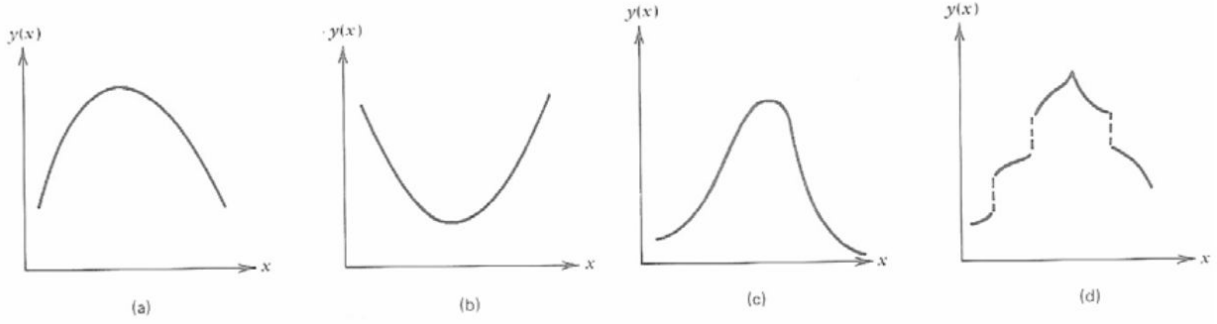
- Una función es unimodal en un intervalo [a, b] si tiene exactamente un máximo o un mínimo en ese intervalo.

Formalmente, una función es unimodal si existe un punto m tal que:

- La función es estrictamente creciente/decreciente para x en [a, m].
- La función es estrictamente decreciente/creciente para x en [m, b].

Búsqueda Ternaria

- La búsqueda ternaria se utiliza para encontrar el máximo o el mínimo de una función unimodal.
- La búsqueda binaria divide el rango en dos partes iguales.
- La búsqueda ternaria divide el rango en tres partes.

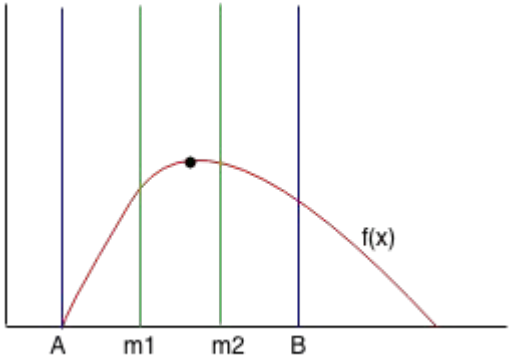


Examples of unimodal functions $y(x)$.

Búsqueda Ternaria

Proceso:

1. Dividir el rango en tres partes.
2. Evaluar la función en dos puntos, m1 y m2, que dividen el rango en tres partes iguales.
3. Comparar los valores de la función en m1 y m2.
4. Reducir el rango de búsqueda a dos tercios del rango original dependiendo de la comparación.



Búsqueda Ternaria Código

```
double ternary_search(double l, double r) {
    double eps = 1e-9;              //set the error limit here
    while (r - l > eps) {
        double m1 = l + (r - l) / 3;
        double m2 = r - (r - l) / 3;
        double f1 = f(m1);      //evaluates the function at m1
        double f2 = f(m2);      //evaluates the function at m2
        if (f1 < f2)
            l = m1;
        else
            r = m2;
    }
    return f(l);                    //return the maximum of f(x) in [l, r]
}
```

Búsqueda ternaria en enteros

```
int ternary_search(int l, int r) {
    int res = -1;
    while(l < r) {
        int mid = (l + r) / 2;
        if(f(mid) < f(mid + 1)) {
            l = mid + 1, res = mid + 1;
        } else {
            r = mid, res = mid;
        }
    }
    return res;
}
```